

LeBlanc: Re-evaluating Security Scaffolding for Contemporary Code LLMs

B.Tech Capstone Progress and Research Direction Report

22 August 2026

Abstract

LeBlanc is a secure LLM-code-generation research platform and Model Context Protocol (MCP) tool. It evaluates whether two established interventions—CWE-aware prompt guidance and scanner-guided iterative repair—continue to improve the security of code generated by contemporary models. Our preliminary pilot across the functionally testable prompt subset has motivated a sharper contribution: rather than assuming these interventions remain beneficial, we will measure where their benefit persists, where it declines, and when repair produces code that is scanner-clean but functionally degraded. The outcome is a reproducible empirical study and a practical MCP security workflow for coding agents.

1 Motivation and Research Opportunity

LLMs are increasingly used to write application code. Earlier studies demonstrated that vulnerability-aware instructions and security feedback can improve the security of generated code. However, these results were obtained with earlier model generations, particular benchmarks, and differing evaluation procedures. As code models improve rapidly, a practical question remains open: *does this security scaffolding still provide enough marginal value to justify using it?*

LeBlanc addresses this question directly. The project is strategically positioned as a modern re-evaluation of established methods, not as a claim that it invented them. This is valuable because both positive and negative results produce useful practitioner guidance: organisations need to know when a security layer should be invoked and when it merely adds latency and complexity.

2 What Has Been Accomplished

- A complete Python pipeline has been implemented: prompt analysis, CWE-aware enrichment, LLM generation, dual static analysis using Bandit and Semgrep, iterative repair, and functional smoke testing.
- A curated 50-prompt Python dataset has been frozen: 10 realistic original tasks and 40 SecurityEval-derived tasks, distributed across security categories.
- Twenty prompts have executable reference-backed smoke tests, enabling the project to measure both security and basic functional preservation.
- A batch runner, experiment database, metrics layer, and dashboard have been built for repeatable collection and presentation.
- An MCP server has been implemented to expose the pipeline to compatible coding agents as reusable tools.
- A preliminary pilot has been conducted over the 20 testable tasks and three models, comprising approximately 600 LLM interactions when generation and repair calls are included.

3 Preliminary Insight and Strategic Pivot

The pilot indicates that recent models may already produce secure standard patterns for many short, well-known security tasks. In such cases, explicit warnings and iterative repair may offer little additional improvement. This is an important result direction, not a weakness: it tests whether a technique that was useful for earlier models remains useful today.

The revised paper will therefore test the following proposition:

Legacy security scaffolding has a diminishing marginal benefit as code LLM capability improves, while scanner-guided repair can still be valuable for selected task categories and weaker models.

LeBlanc will not claim that earlier papers were false. Instead, it will rigorously challenge their generalisability across time: a result established on earlier models should be re-measured on contemporary models before it is treated as universal guidance.

4 Prior Work Being Re-evaluated

Study	Established contribution	LeBlanc re-evaluation
SecuCoGen [1]	CWE-aware information and secure code-generation dataset	Does prompt-level vulnerability guidance still improve current models beyond their unaided baseline?
Guiding AI to Fix Its Own Flaws [2]	Vulnerability hints and feedback-based repair evaluation	Does a repeatable scanner-to-repair loop still improve outcomes, and for which models?
CodeSecEval [3]	Secure generation and repair benchmark	Can a single, reproducible pipeline compare generation, enrichment, and repair on a dated model fleet?
CODEGUARD+ [4]	Security must be assessed alongside correctness	How often do scanner-clean repairs fail basic functional tests?
Copilot replication [7]	Security behaviour changes as code assistants evolve	How large is the change in scaffolding benefit across model eras?

HexaCoder and LPO/DiSCo are important contrasts: they improve models through training or fine-tuning, whereas LeBlanc evaluates inference-time, API-only scaffolding that can be applied to closed models [5, 6].

5 Research Questions and Hypotheses

1. **RQ1: Baseline security progress.** How does the vulnerability rate of current models compare with older/smaller baselines on identical tasks? **H1:** current models have lower baseline vulnerability rates.
2. **RQ2: Enrichment value decay.** Does CWE-aware enrichment reduce vulnerability rates, and does the size of this reduction decline for newer models? **H2:** enrichment helps older models more than contemporary models.
3. **RQ3: Repair-loop value.** What proportion of initially vulnerable generations become scanner-clean after up to three repair iterations? **H3:** repair remains selectively useful where a concrete scanner finding exists.

4. **RQ4: Repair inflation.** How often does a scanner-clean repair fail its functional smoke test? **H4:** static cleanliness alone overstates practical repair success.
5. **RQ5: Deployment rule.** Which model/task combinations justify invoking the MCP workflow in practice?

6 Experimental Plan

The final study will use a frozen, dated model fleet with older/smaller baselines, 2024-era capable comparators, and current 2026 models from multiple providers. Exact model identifiers, access dates, decoding configuration, scanner versions, and price information will be recorded in an experiment manifest before data collection.

For each prompt-model pair, LeBlanc will compare three matched modes:

1. **plain:** direct generation without LeBlanc intervention;
2. **enriched:** CWE-aware prompt guidance before generation; and
3. **enriched_repair:** enrichment plus scanner-guided iterative repair.

Each outcome will record extraction validity, scanner findings, repair iterations, functional-test result where available, and the combined scanner-clean-and-functional verdict. Results will be reported with uncertainty intervals and paired comparisons, not merely raw vulnerability counts.

7 MCP System Deliverable

The MCP server turns the research workflow into a usable coding-agent integration. A compatible assistant can:

- analyse a coding request for likely CWE risks before code is generated;
- scan a generated response or an approved workspace path using Bandit and Semgrep;
- receive structured, category-level findings; and
- request an optional scanner-guided repair, subject to testing and human review.

The MCP deliverable demonstrates applied value regardless of the final research outcome. If the study finds selective benefit, the MCP can invoke scaffolding only for the relevant models and task categories. If benefits are small for current models, it remains valuable as a security-audit and explanation tool rather than claiming to be an automatic security guarantee.

8 Planned Outputs and Next Steps

1. Consolidate the preliminary pilot data into the experiment database and produce a reproducible pilot report.
2. Freeze and preflight the final model fleet; record all provider and configuration details.
3. Complete the full matched experiment matrix and perform a false-positive audit of static findings.
4. Generate figures for baseline rates, enrichment effect, repair convergence, residual-risk categories, and repair inflation.
5. Finalise the MCP safety controls, especially workspace restriction for path scanning and advisory-only repair output.
6. Produce the paper, demonstration dashboard, MCP setup guide, dataset, and experiment artefacts.

9 Expected Contribution

LeBlanc will contribute a timely answer to a question that current secure-code-generation literature leaves open: *when should modern coding agents still use legacy prompt-and-repair security scaffolding?* The project combines a reproducible evaluation harness, correctness-aware repair analysis, and an MCP implementation that translates empirical findings into a practical agent workflow.

References

- [1] Wang et al. “Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on Vulnerability Mitigation.” SecuCoGen, 2024. [docs/researchdocs/Two.pdf](#)
- [2] Yan et al. “Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation.” 2025. [docs/researchdocs/One.pdf](#)
- [3] Wang et al. “Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval.” [docs/researchdocs/Three.pdf](#)
- [4] Fu et al. “Constrained Decoding for Secure Code Generation.” CODEGUARD+. [docs/researchdocs/Five.pdf](#)
- [5] Hajipour et al. “HexaCoder: Secure Code Generation via Oracle-Guided Synthetic Training Data.” [docs/researchdocs/Seven.pdf](#)
- [6] Hasan et al. “Teaching an Old LLM Secure Coding: Localized Preference Optimization on Distilled Preferences.” [docs/researchdocs/Four.pdf](#)
- [7] Majdinasab et al. “Assessing the Security of GitHub Copilot’s Generated Code: A Targeted Replication Study.” [docs/researchdocs/Eight.pdf](#)